Assignment No. 5
EECS 348
Software Engineering I
Due: 11:59 PM, Thursday, October 29, 2026

Deliverable (submitted to Canvas): URL to your GitHub repository

- Make sure the repository is private but provides access to your GTA so they are able to access and grade it. Instructions for making your repository private are located here: https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility#changing-a-repositorys-visibility
- Use the following naming convention for your GitHub repository:
  - Repository: Fall2026_EECS348_Assignment_5
  - Folder: Firstname_Lastname_Assignment_5
- Your GitHub repository should include only these files:
  - Source code named: Assignment5.cpp (single file, no multiple files)
  - Final cycle server executable file named: Assignment5.exe
  - PDF of your GenAI analysis named: Assignment5GenAI.pdf
  - UML Use Case Diagram named: Assignment5UseCase.pdf
  - Suduko puzzle files: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt
- Notes:
  - Failure to use the proper naming convention may result in a zero for the assignment.
  - Including files other than those specified above may result in a zero for the assignment.
  - The EECS Cycle servers (or the lab machines) are the only grading environment.
  - Please note that the same code may produce different outputs in different environments, especially for C/C++ programs that require compilation.
  - Since we cannot access your personal environment, grading will be based solely on the Cycle servers (or the lab machines).
  - You must ensure that your submission works correctly in the allowed grading environment.
  - Appeals for regrading based on the code working in your own environment will not be accepted.
  - This policy applies to all labs and programming assignments for this course.
- You must also provide the source code and Use Case Diagram for your Assignment 4 in a zip file to another student as described below via email by Friday, October 16, 11:59 PM. Failure to do so will result in a zero for Assignment 5. Send as a zip file because the email system may delete attached C++ code.

Assignment:

1. For this assignment, you will modify another student's Assignment 4. The file named, "Assignments for Assignment 5.pdf", lists whose Assignment 4 you will modify and who will modify your Assignment 4.
    - If you do not see your name, contact me immediately.
    - If you do not receive the Assignment 4 you are supposed to modify by Friday, October 16, 11:59 PM. Contact your GTA.
2. Use two GenAI's to modify the Assignment 4 code you receive as follows (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
    - Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.
    - Each solutionN.txt file should include the following:
        - If no solutions are found, a single line: "No solution found."
        - If solutions are found, it should include all solutions followed by a single line: "Solutions found: n", where n is the number of solutions found.
        - The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.
    - You must modify the Assignment 4 you receive, not your Assignment 4.
    - Do not replace the entire file with your code. Your GTA will use the Github Blame button to check your modifications.
    - Add your name to the Profile Comments as "Revised by:" along with a revision date.
    - You must get all required functionality from Assignment 4 working, even if the student you inherited it from did not.
3. Analyze the two GenAI programs. The analysis must include:
    a. Name of two GenAI programs
    b. How you accessed them (e.g., Google query, CoPilot on my computer)
    c. Prompt used. (You must use the same prompt for both GenAIs.)
    d. Generated code from both GenAIs.
    e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.
    f. Select one to use as the basis for this assignment and justify why you picked it.
    g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.
4. Make the changes identified in 3g above.
5. Compile and debug your program.
6. Push both your source code, final cycle server executable file, and PDF of your GenAI analysis to your GitHub repository.
7. Use a GenAI of your choice to modify the UML Use Case Diagram you received for this assignment with the level of detail given in the first UML Use Case Modeling lecture:
    a. Identify/discover a use case
    b. Provide a brief description for the use case

c. Provide a step-by-step flow (outline) of the events for the use case
d. Provide a detailed description for the use case

Do not include any analysis of the GenAI Use Case Diagram in your GenAI analysis and if it does not provide the level of detail required, you will need to provide it.

Rubric for Program Comments		
Exceeds Expectations (90-100%)	**Meets Expectations (80-89%)**	**Unsatisfactory (0-79%)**
Software is adequately commented with prologue comments, comments summarizing major blocks of code, and comments on every line.	Prologue comments are present but missing some items or some major blocks of code are not commented or there are inadequate comments on each line.	Prologue comments are missing all together or there are no comments on major blocks of code or there are very few comments on each line.

Adequate Prologue Comments:

- Name of program contained in the file (e.g., EECS 348 Assignment 1)
- Brief description of the program, e.g., CEO Email prioritization program
- Inputs e.g., test file labeled Assignment1_Test_File.txt
- Output, e.g., Next email and number of unread emails
- All collaborators
- Other sources for the code ChatGPT, stackOverflow, etc.
- Author's full name
- Creation date: The date you first create the file, i.e., the date you write this comment

Adequate comments summarizing major blocks of code and comments on every line:

- Provide comments that explain what each line of code is doing.
- You may comment each line of code and/or provide a multi-line comment that explains what a group of lines does.
- Multi-line comments should be detailed enough that it is clear what each line of code is doing.
- Each block of code must indicate whether you authored the code, you obtained it from one of the sources listed in the prolog, or one of your collaborators authored the code, or if it was a combination of all of these.

Collaboration and other sources for code:

- When you collaborate with other students or use other sources for the code (e.g., ChatGPT, stackOverflow):
  - Your comments must be significantly different from your collaborators.
  - More scrutiny will be applied to grading your comments in particular explaining the code "in your own words", not the source's comments (e.g., ChatGPT's comments).

- Failure to identify collaborators or other sources of code will not only result in a 0 on the assignment but will be considered an act of Academic Misconduct.
- Students who violate conduct policies will be subject to severe penalties, up through and including dismissal from the School of Engineering.